

UNIVERSITY OF THE PACIFIC

APPLIED LINEAR ALGEBRA

MATH 145

Recovering the Signal of the Lost Operative

Author(s):

Justin M. BARBER

Angel MEJIA

Guy C. MOORE

Latimer D. HARRIS-WARD

Supervisor:

Dr. Kevin LAMB

September 22, 2026



CONTENTS

1	Basic Filtering Function	2
2	Initial Filtering Ideas.	3
3	Comparison to Non-Jammed Signal	12
4	Wave Interference & Convolution Study.	12
5	Secondary Filtering Results: Towards Envelope Interference Filtering	14
6	Analyzing Different Filtering Methods, Modifying Frequencies, and Difficulties Encountered	22
7	Conclusion	26

1. BASIC FILTERING FUNCTION

Below is a basic filtering function for removing background noise of a signal.

```
1 function [filtered_data] = FilterSignal(data,fs,max_freq_str,min_freq_str)
2 n = length(data);
3 DFT = fft(data);
4 f = (0:n-1)*(fs/n);
5
6 DFT_low_ind = find(abs(DFT)>max_freq_str & abs(DFT)<min_freq_str);
7 DFT_low = zeros(size(DFT));
8 DFT_low(DFT_low_ind) = DFT(DFT_low_ind);
9
10 filtered_data = ifft(DFT_low);
11
12 figure
13 plot(f,abs(DFT_low),'b-')
14 title('Removed Very Low and Very High Frequencies of DFT','FontSize',14)
15 figure
16 plot(f,abs(DFT),'b-')
17 title('DFT of Original Signal','FontSize',14)
18 figure
19 plot(filtered_data,'b-')
20 title('Filtered Data','FontSize',14)
21 end
```

Using the test functions given in the project, this function recreates the original signal almost perfectly. However, this function only works if the noise is strictly weaker or stronger than the original signal. Since we knew the strength of frequencies to keep or remove, the filtering was simple. If instead the noise had a similar strength to the original signal, knowing what frequency strengths to keep would become difficult. Also to note, this function does not replicate the original function exactly, since only frequencies with unwanted strengths would be removed. The frequencies of the original signal would still be modified slightly by

the noise, which the function does not account for. However, since the added noise is weak, this does affect the filtered signal too greatly. This would again cause another issue if the noise were too strong.

2. INITIAL FILTERING IDEAS

The first idea we had upon receiving the jammed signal was to remove frequencies outside of the vocal range. We determined this range to be 100 to 3000 Hz. However, even after filtering all frequencies outside of this range, the noise still persists. We tested saving different ranges of frequencies, and noticed an interesting feature: the resulting wave seemed to always be the same. No matter what frequency window we choose, the wave pattern keeps the same shape.

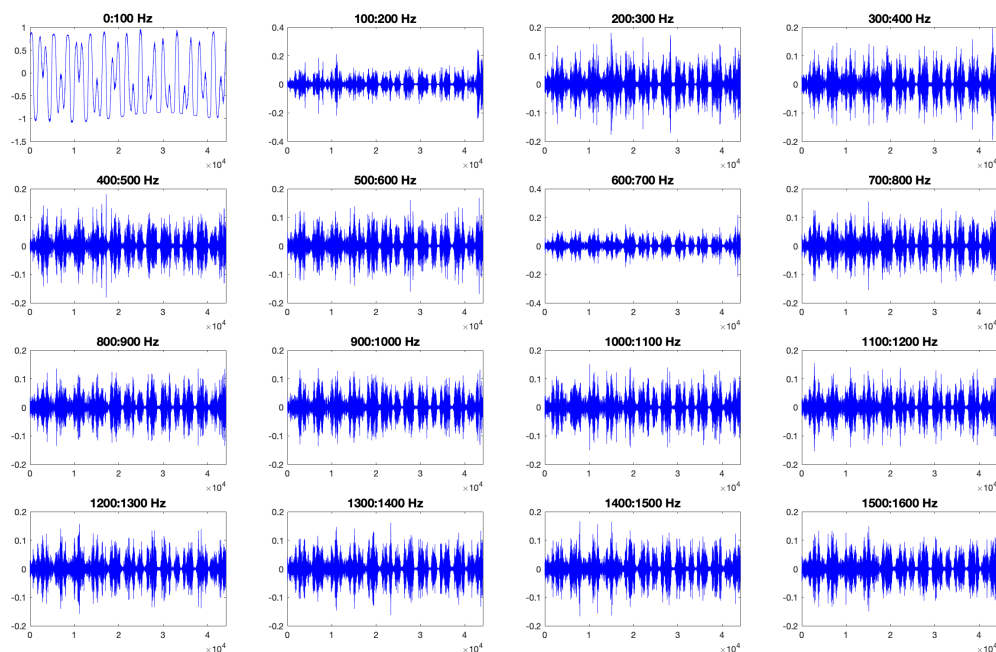
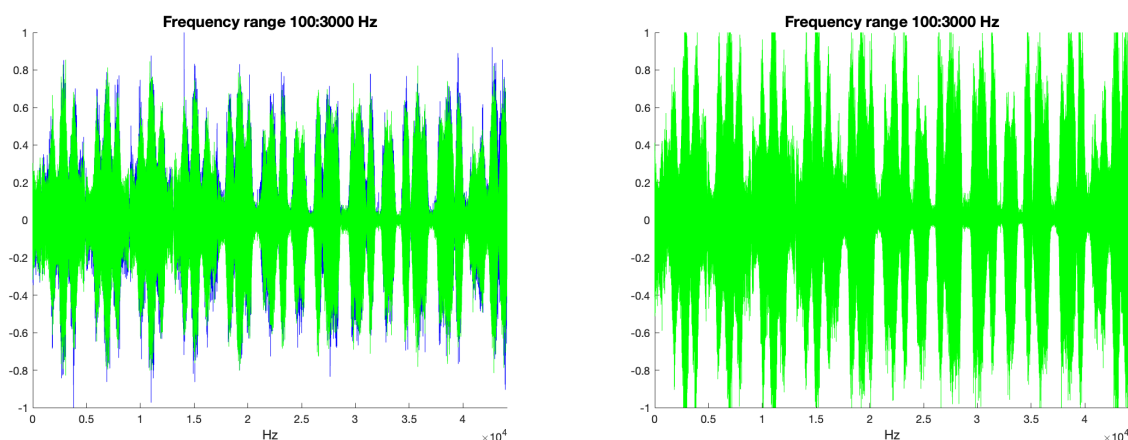


Figure 1: Resulting waves of a 100 Hz frequency window.

There are several notable facts about this pattern. As the frequency window increases, the amplitude of the resulting wave increases. Furthermore, the pattern seems to only form

with frequency windows larger than 50-60 Hz; lower frequency windows vary too wildly to form the pattern. This pattern also corresponds to the noise we hear in the signal. Each of the peaks in the wave match with the rhythmic noise heard in the recording.

We thought we could use this to our advantage since this pattern persisted in every frequency range. We went into the high frequencies of the signal where no voice should exist, averaged 100 waves of these frequencies to find the average pattern, then subtracted this wave from the wave formed of vocal frequency ranges. However, we noticed that average high frequency wave did not simply subtract from the vocal wave. Although they appeared near identical, subtracting the wave seemed to make the wave worse. We believe this is due to the highly oscillatory nature of the waves; perhaps the upper points are subtracting lower points, and vice versa.



(a) The average high frequency wave (green) overlaid the vocal frequencies (blue). (b) The vocal frequency wave subtracted by the average wave.

Figure 2

We then tried repeating this process but with the Discrete Fourier Transform (DFT) instead, but similar problems arose.

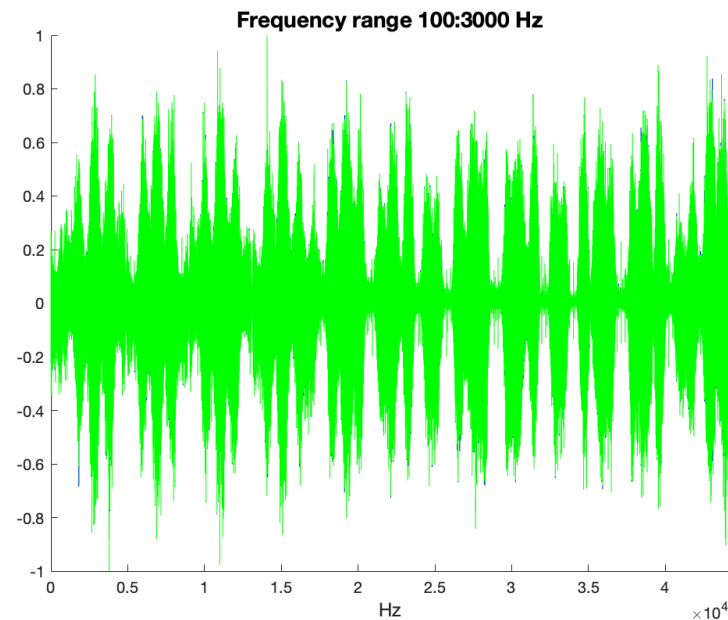
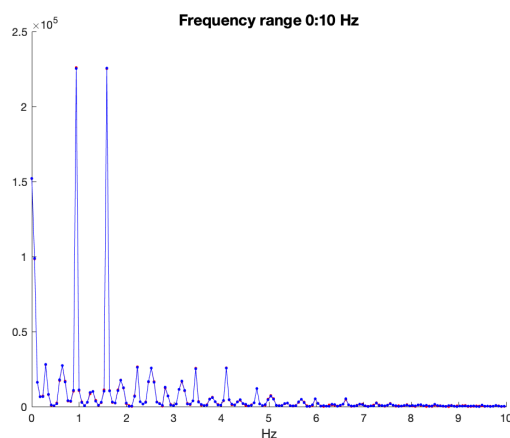
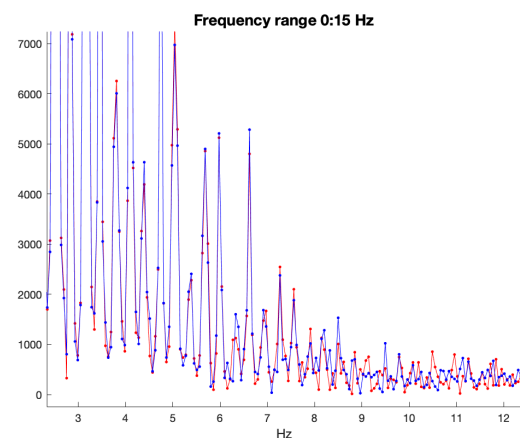


Figure 3: Wave formed by subtracting the average high DFT range from the vocal DFT range.

We abandoned this idea and then thought of a new one: perhaps the cause for the persisting wave pattern, i.e. the noise, is the low frequency waves. These frequencies, specifically from 0 to 10 Hz, seem to be the most powerful frequencies of the noise.



(a) DFT plot of 0 to 10 Hz.



(b) DFT plot flattening out past 10 Hz.

Figure 4

We had the idea that perhaps multiples of these low frequencies would be the cause of the noise, similar to overtones in music. So then we tried to remove frequencies that were integer multiples of frequencies less than 10 Hz. However, the resulting wave did not remove much noise and only seemed to flatten the wave somewhat. The voice is still unintelligible.

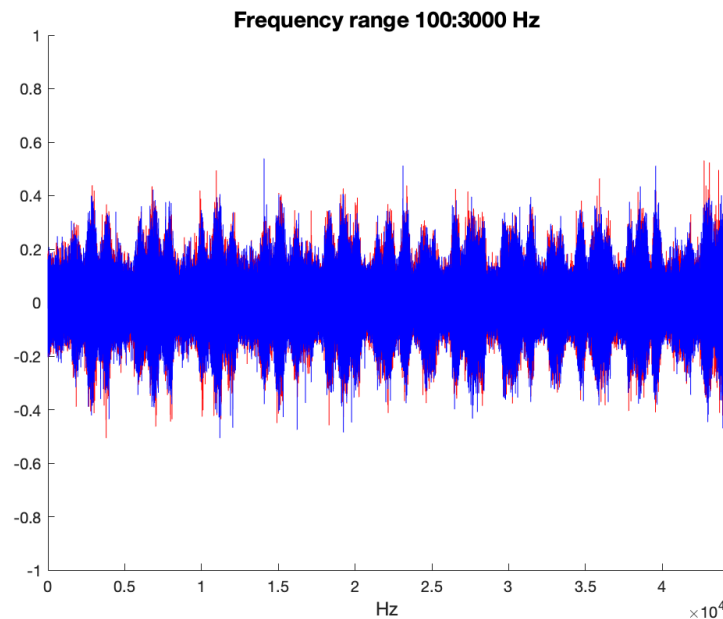


Figure 5: Wave with removed integer multiples of low frequencies.

We also tried what the `FilterSignal` function from the earlier section did as well, but we could not remove any of the rhythmic noise without removing the clear part of the voice near the end of the recording that said "do not...!"

At this point, we begun thinking about how to remove the rhythmic noise. This noise seems to follow the same wave pattern despite the frequencies composing it. This would only be possible if there were an envelope that restricted the waves inside. We thought once we found this envelope we could determine the noise and filter it out, but the problem is that this envelope does not exist as a separate low frequency wave. The wave pattern still exists even if the low frequencies have been removed, meaning the envelope manifests from a sum of waves. This makes determining the envelope, and therefore the noise, much more difficult than simply identifying a low frequency wave.

Below is the code used for these results.

```
1 % load data
2 [y,Fs] = audioread('Message.wav');
3 player = audioplayer(y,Fs);
4
5 % create frequency and DFT
6 n = length(y);
7 DFT = fft(y);
8 f = (0:n-1)*(Fs/n);
9
10 % find average pattern at high frequencies
11 df = f_lim(2) - f_lim(1);
12 N = 100;
13 yy = zeros(size(DFT));
14 yyy = zeros(size(DFT,1),size(DFT,2),N);
15 avg_DFT = zeros(length(f_range),2);
16 DFTs = zeros(length(f_range),2,N);
17 for i=1:N
18     init_f = 30000 + 10*i;
19     temp_f_lim = [init_f init_f + df];
20     temp_f_range = find(f >= temp_f_lim(1) & f <= temp_f_lim(2));
21     temp_DFT = zeros(size(DFT));
22     temp_DFT(temp_f_range,:) = DFT(temp_f_range,:);
23     temp_DFT(n-(temp_f_range),:) = DFT(n-(temp_f_range),:);
24
25     temp_avg_DFT = DFT(temp_f_range,:);
26     if length(temp_avg_DFT) < length(avg_DFT)
27         temp_avg_DFT = [temp_avg_DFT; 0 0];
28     end
29     DFTs(:, :, i) = temp_avg_DFT;
30     avg_DFT = avg_DFT + temp_avg_DFT;
31
32     temp_y = ifft(temp_DFT);
```

```
33     yy = yy + temp_y;
34     yyy(:, :, i) = temp_y;
35 end
36 yy = yy./N;
37 avg_DFT = avg_DFT./N;
38 playery = audioplayer(yy,Fs);
39
40 DFT3 = zeros(size(DFT));
41 DFT3(f_range, :) = avg_DFT;
42 DFT3(n-(f_range), :) = avg_DFT;
43 y3 = ifft(DFT2-DFT3);
44
45 % create wave of 0:10 Hz
46 low_range = find(f<=10);
47 low_DFT = zeros(size(DFT));
48 low_DFT(low_range, :) = DFT(low_range, :);
49 low_DFT(n-(low_range), :) = DFT(n-(low_range), :);
50 low_y = ifft(low_DFT);
51
52 % remove integer multiples of large spikes
53 N = 4;
54 % [I1, I2] = maxk(abs(DFT(:,1)),N);
55 % [I1, I2] = maxk(abs(DFT(:,2)),N);
56 I1 = find((f<10 & f>2) | (f>f(end)-10) & f<(f(end)-2));
57 I2 = I1;
58 for i=1:length(I1)
59     if (I1(i) < n/2) && (I2(i) < n/2)
60         mult1 = 1:floor((n/2)/I1(i));
61         mult2 = 1:floor((n/2)/I2(i));
62         DFT(I1(i)*mult1,1) = 0;
63         DFT(I2(i)*mult2,2) = 0;
64     elseif (I1(i) > n/2) && (I2(i) > n/2)
65         mult1 = 1:floor((n/2)/(n-I1(i)));
66         mult2 = 1:floor((n/2)/(n-I2(i)));
```

```
67         DFT(n-(n-I1(i))*mult1),1) = 0;
68         DFT(n-(n-I2(i))*mult2),2) = 0;
69     else
70         disp('something wrong')
71     end
72 end
73
74 % remove frequencies with coefficients that are too low or high
75 DFT_lh_ind = find(abs(DFT)>200);
76 DFT_lh = zeros(size(DFT));
77 DFT_lh(DFT_lh_ind) = DFT(DFT_lh_ind);
78
79 % % do a moving mean
80 % ymean = movmean(y,5);
81 % dftmean = fft(ymean);
82 % dftmean2 = zeros(size(dftmean));
83 % dftmean2(4100:(n/2),:) = dftmean(4100:(n/2),:);
84 % dftmean2(n-(4100:(n/2)),:) = dftmean(n-(4100:(n/2)),:);
85 % ymean2 = ifft(dftmean2);
86 % plot(real(ymean2))
87 % figure
88 % plot(abs(dftmean2));
89 % ylim([0 1000])
90 % xlim([0 n/2])
91 % playermean = audioplayer(real(ymean2),Fs);
92
93 % remove frequencies outside of range
94 f_lim = [0 10]; %vocal range 100 to 3000 or 300 to 3000
95 f_range = find(f>f_lim(1) & f<f_lim(2));
96 DFT2 = zeros(size(DFT));
97 DFT2(f_range,:) = DFT(f_range,:);
98 DFT2(n-(f_range),:) = DFT(n-(f_range),:);
99
100 % reconstruct wave
```

```
101 y2 = ifft(DFT2);
102 player2 = audioplayer(y2,Fs);
103
104 % plot DFT and reconstructed wave
105 figure
106 hold on
107 plot(f,abs(DFT2(:,1)), 'r.-')
108 plot(f,abs(DFT2(:,2)), 'b.-')
109 %ylim([0 3000])
110 xlim([f_lim(1) f_lim(2)])
111 title(sprintf('Frequency range %d:%d Hz',f_lim(1),f_lim(2)), 'FontSize',14)
112 xlabel('Hz', 'FontSize',12)
113
114 % figure
115 % hold on
116 % plot(f,abs(DFT(:,1)), 'r.-',f,abs(DFT(:,2)), 'b.-')
117 % %ylim([0 3000])
118 % xlim([f(end)-f_lim(2) f(end)-f_lim(1)])
119 % title(sprintf('Frequency range ...
    %g:%g',f(end)-f_lim(2),f(end)-f_lim(1)), 'FontSize',14)
120 % xlabel('Hz', 'FontSize',12)
121
122 figure
123 hold on
124 plot(f,real(y2(:,1)), 'r-',f,real(y2(:,2)), 'b-')
125 %plot(f,real(yy), 'g-')
126 % ylim([-0.2 0.8])
127 % xlim([1500 4000])
128 ylim([-1 1])
129 xlim([f(1) f(end)]);
130 title(sprintf('Frequency range %d:%d Hz',f_lim(1),f_lim(2)), 'FontSize',14)
131 xlabel('Hz', 'FontSize',12)
132
133 %%
```

```
134
135 % plot sound waves as frequency increases
136 f1 = figure;
137 f2 = figure;
138 for i=1:16
139     DFT = fft(y);
140     f_lim = [100*(i-1) 100*i];
141     f_range = find(f >= f_lim(1) & f <= f_lim(2));
142     DFT2 = zeros(size(DFT));
143     DFT2(f_range,:) = DFT(f_range,:);
144     DFT2(n-(f_range),:) = DFT(n-(f_range),:);
145     y2 = ifft(DFT2);
146
147     figure(f1)
148     subplot(4,4,i)
149     plot(f,real(y2(:,1)),'b-')
150 %     ylim([-0.2 0.2])
151     xlim([f(1) f(end)]);
152     title(sprintf('%d:%d Hz',f_lim(1),f_lim(2)),'FontSize',14)
153
154     figure(f2)
155     subplot(4,4,i)
156     hold on
157     plot(f,abs(DFT2(:,1)),'r.-')
158     plot(f,abs(DFT2(:,2)),'b.-')
159     xlim([f_lim(1) f_lim(2)])
160 %     ylim([0 1000])
161     title(sprintf('%d:%d Hz',f_lim(1),f_lim(2)),'FontSize',14)
162 end
```

3. COMPARISON TO NON-JAMMED SIGNAL

To study the spectrum of a voice, audio recordings of members of the group were recorded, and examined in the following way in comparison to the jammed signal:

- Compared original message to an audio recording of the same time length
- Compared Gauss low-pass filter to high frequency spikes to try to identify spike patterns
- Used Gauss graph to find patterns at both high and low frequencies
- In addition to another “pure” vocal recording, we examined the last half second of the assignment recording, during which the frantic words “Do Not... !” is audible. Using this filtered sample, we tried to perform both:
 - Windowed projection of the jammed signal onto this “cleaned” signal, in order to extract desired frequencies
 - Convolution of the jammed signal with the sample, by finding the windowed product of the DFT with the original signal

Early on we decided to record a message that way we could use it’s graph and compare that to the original message left by the agent. We initially used a recording of Latimer, the only problem was that the message was under 10 seconds as compared to 17 from the original message. It didn’t seem logical to us to compare the two given the significant time difference, so we decided to use a 17 second recording of Angel instead. Our ideas involving the use of a Gauss graph were thought of early on in the project and were later expanded on. Although somewhat illustrative, due to the difference between vocal frequency profiles between each recording, these ideas were shortly dropped as results proved insignificant.

4. WAVE INTERFERENCE & CONVOLUTION STUDY

In lieu of the previous sections, several different approaches were explored in depth. The progression of discoveries followed the following form:

- We isolated the frequency range of the human voice, removing waves with frequencies above or below this region
- Fourier components with a large magnitude were originally suspected to correspond to the jamming signal, as the jamming signal would dominate the original signal. However, we found that simply removing these components resulted in significant loss of the originally detected voice, implying a interference entanglement pattern between the jamming signal and peak vocal modes
- We identified a “beat” interference pattern present in all signals, including the filtered signal

Consider the following derivation for the interference between two sinusoids of equal magnitude, where $\Delta\omega = \omega_i - \omega_e$ and $\hat{\omega} = \omega_i + \omega_e$:

$$\begin{aligned}\cos(\hat{\omega}t) + \cos(\Delta\omega t) &= \frac{1}{2} (e^{i\hat{\omega}t} + e^{-i\hat{\omega}t} + e^{i\Delta\omega t} + e^{-i\Delta\omega t}) = \frac{1}{2} (e^{i\omega_e t} + e^{-i\omega_e t}) (e^{i\omega_i t} + e^{-i\omega_i t}) \\ &= 2 \cos(\omega_e t) \cdot \cos(\omega_i t)\end{aligned}$$

Where in this case, ω_e is the “beat” or envelope frequency, and ω_i is the interior or mean frequency of the two waves causing the interference.

Originally, we believed that the signal could be unfiltered by identifying the kernel of the jamming operation, which we will denote as $\mathcal{J}$,

$$\mathcal{J}[f(n)] = g(n)$$

where $f(n)$ is the original signal and $g(n)$ is the modified signal with jamming added. Assuming that $\mathcal{J}$ is a linear operation, and we know the hypothetical kernel of $\mathcal{J}$, $h(n)$ as the linear response to a Kronecker delta forcing term (one for $n = 0$, zero elsewhere),

$$\mathcal{J}[h(n)] = \delta(n)$$

It can be shown that the following is true

$$f(n) = (h * g)(n)$$

and therefore the solution to the linear system is simply the resulting signal convolved with the kernel of the operation. Furthermore, it can also be shown that the following is true: taking the Discrete Fourier Transform (DFT) of the original signal gives

$$\widehat{f}(k) = \widehat{h * g}(k) = \widehat{h}(k)\widehat{g}(k)$$

Therefore, in Fourier space, convolution involves the element-wise multiplication of the Fourier coefficients of the signal.

To remove the high frequency components of the noise, we performed a moving average on the signal, in order to smooth it out. This is equivalent to performing a low-pass filter. This can be reasoned using the above equations, because a moving average is equivalent to convolving the signal with a pulse wave in real space. The discrete Fourier transform of a pulse wave (which is the kernel of the moving average operation), is a *sinc* function, which can be expressed in the form of $\widehat{h}(k) = \text{sinc}(k) = \frac{\sin(k)}{k}$.

5. SECONDARY FILTERING RESULTS: TOWARDS ENVELOPE

INTERFERENCE FILTERING

- Examining the initial signal, we see that we can first smooth the signal due to the sharpness induced from limiting the signal between -1 and +1.

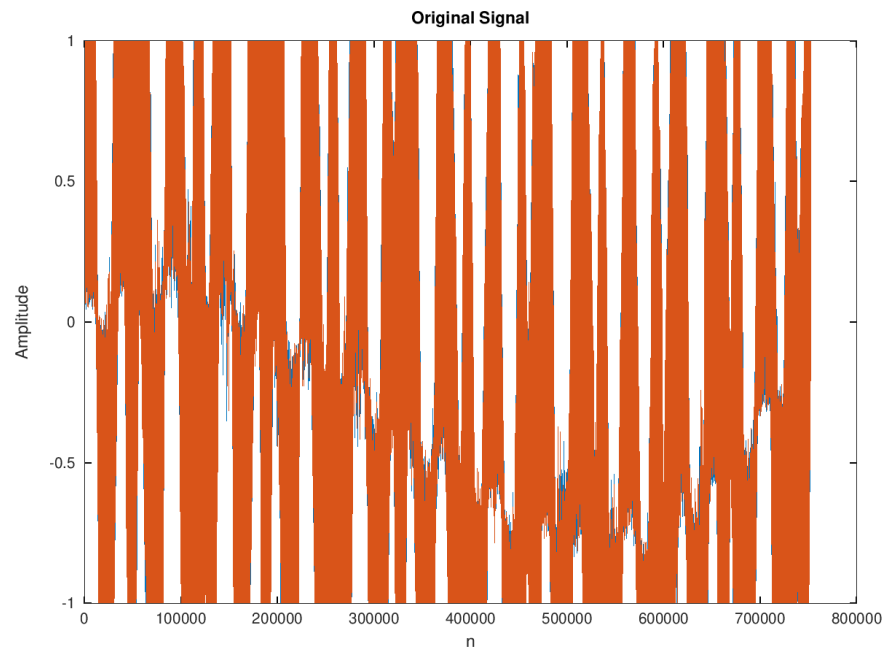


Figure 6

- In order to smooth the signal, a moving average is imposed on the original signal with a width of 20. As we showed previously, this is equivalent to a low-pass filter. A custom moving average filter was implemented for this project. A sparse diagonal Toeplitz matrix was constructed, which is used to operate on the signal to produce a new, smoother signal.

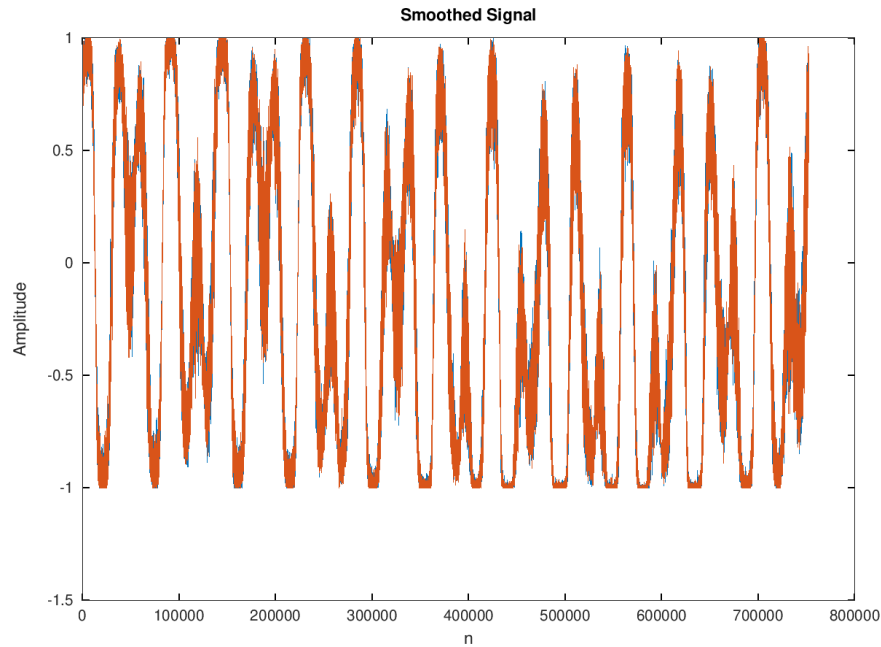


Figure 7

- Next, based on the logic presented earlier in this section, we observed an interference pattern in the signal, which is for all intents and purposes independent from the range of frequencies selected. In the code presented, we (manually, for the time being) identify peaks spaced at a frequency near to the observed beat frequency. Let Δk correspond to the apparent beat frequency of the signal, then if we identify two peaks $\hat{f}_{k-\Delta k/2}$ and $\hat{f}_{k+\Delta k/2}$, we perform the following operation to remove the beat effect of the signal

$$\begin{aligned} \hat{f}_{k+\Delta k/2} &= 0 & \hat{f}_{k-\Delta k/2} &= 0 \\ \hat{f}_k &= \hat{f}_k + \frac{1}{2} \left(\hat{f}_{k+\Delta k/2} + \hat{f}_{k-\Delta k/2} \right). \end{aligned}$$

The results of which are shown in the figure below

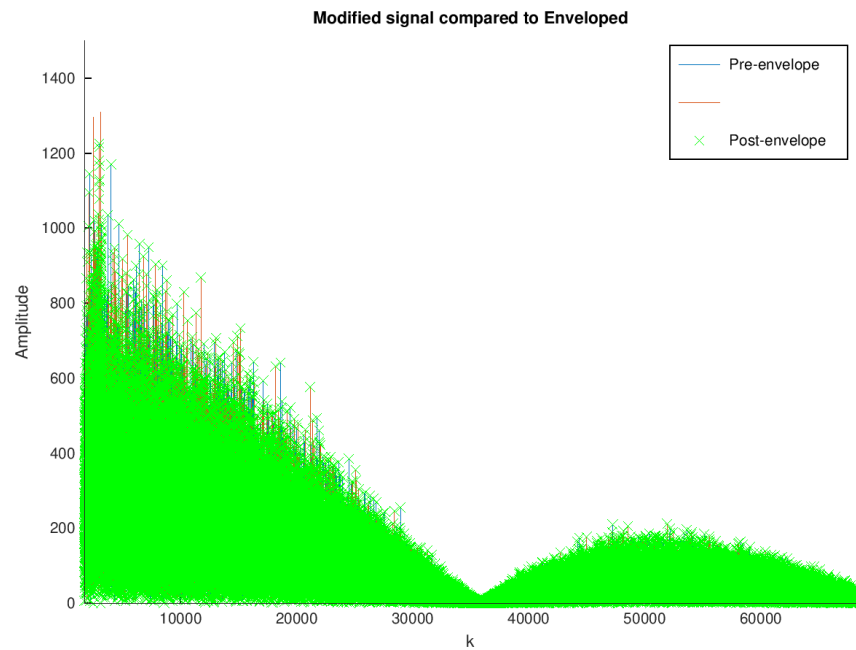


Figure 8

- The figure below is the final DFT of the modified signal. It is apparent that the spectrum resembles a restricted sinc function.

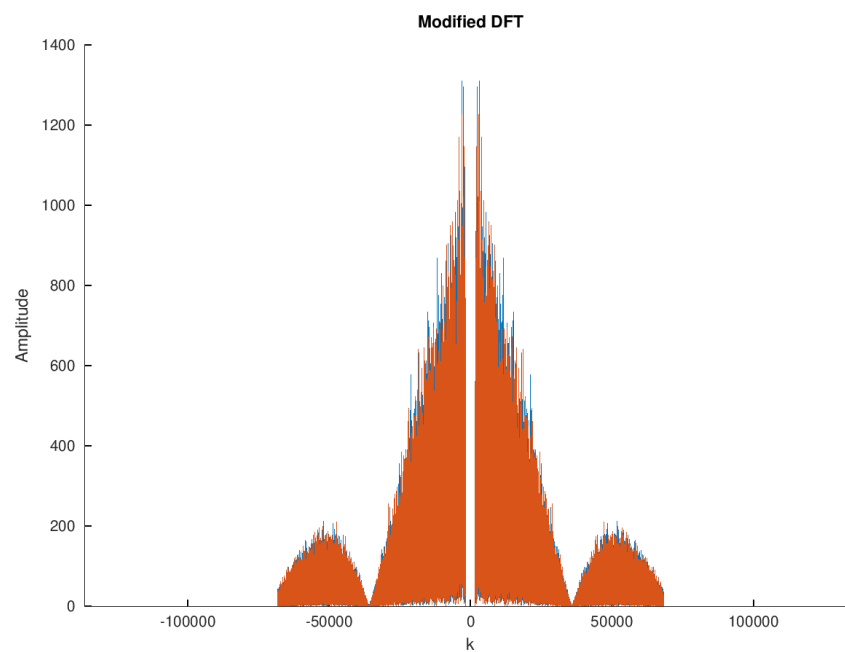


Figure 9

- Lastly, our end result is shown below. As was mentioned earlier in this paper, there still exists a beat pattern that we were unable to remove.

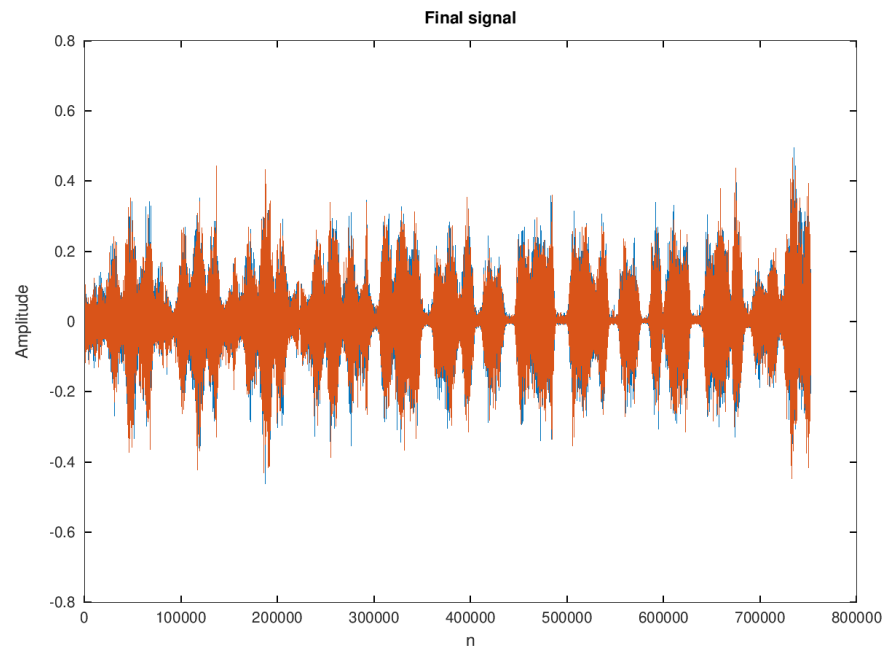


Figure 10

Below is the code used to produce these results, and the filtering progression for reference.

```
1 %% Cleanup
2 clear; clc;
3 close all;
4
5 %% Read in file
6 filename = "Message.wav";
7 [y,Fs] = audioread(filename);
8
9 n = length(y);
10
11 %% Define k-space ordering
12 xk = (-n/2:n/2-1)';
13
```

```
14 %% Truncate and flip indices
15 nhf = n/2+1;
16 xhf = 1:nhf;
17
18 %% Plot initial signal
19 fig_orig = figure;
20 plot(1:n,y)
21 title("Original Signal")
22
23 %% Create moving average sparse matrix
24 nexp = 1;
25 hlfw = 10;
26 win_w = 2*hlfw+1;
27 d = -hlfw:hlfw;
28 B = 1/win_w*ones(n,win_w);
29 A = spdiags(B,d,n,n);
30 for i=1:hlfw
31     A(i,:) = A(i,:)*win_w/nnz(A(i,:));
32     A(end-i+1,:) = A(end-i+1,:)*win_w/nnz(A(end-i+1,:));
33 end
34
35 %% Apply moving average filter
36 y = A^nexp*y;
37
38 %% Plot smoothed signal
39 fig_1 = figure;
40 plot(1:n,y)
41 title("Smoothed Signal")
42
43 %% Perform FFT
44 ydft = fft(y);
45
46 %% Isolate audible frequencies
47 f_lo = 100;
```

```
48 f_width = 4000-f_lo;
49 freq_range = (0+[0 f_width]); % Human voice frequency range
50 k_range = freq_range*(n/Fs);
51 indx = f_lo*(n/Fs); % Window offset
52
53 ydftp = ydft(xhf,:);
54 ydftp((xhf<(k_range(1)+indx) | xhf>(k_range(2)+indx)),:) = 0;
55
56 %% Identify peaks and remove envelope
57 peak_ind = [2507 3122];
58 ydftenv = ydftp;
59 for i=1:size(peak_ind,1)
60     new_ind = round(mean(peak_ind(i,:)));
61     ydftenv(peak_ind,:) = 0;
62     ydftenv(new_ind,:) = ydftenv(new_ind,:) + 0.5*sum(ydft(peak_ind,:));
63 end
64 ydftenv = [ydftenv(1:end,:); flipud(conj(ydftenv(2:end-1,:)))];
65 yenv = ifft(ifftshift(ydftenv));
66
67 figure
68 plot(1:n,yenv)
69
70 %% Set to enveloped
71 y = yenv;
72
73 %% Plot modified spectrum
74 figure
75 hold on
76 plot(xhf,abs((ydft(1:nhf,:))))
77 plot(xhf,abs((ydftenv(1:nhf,:))), 'gx', 'linewidth',1)
78 ylim([0 1500])
79 xlim(indx+k_range)
80
81 %% Restore to full size
```

```
82 ydftp = [ydftp(1:end,:); flipud(conj(ydftp(2:end-1,:)))];
83 ydftp = fftshift(ydftp);
84
85 %% Inverse FFT
86 yp = ifft(fftshift(ydftp));
87
88 %% Perform second smoothing using moving average
89 yp = A^nexp*yp;
90
91 %% Plot modified DFT
92 figure
93 hold on
94 plot(xk,abs(ydftp(:,:)))
95 xlim(2*(indx+k_range(2))*[-1 1])
96
97 %% Plot kernel
98 figure
99 hold on
100 plot(abs(fftshift(fft(kernel)))))
101
102 %% Plot final signal
103 figure
104 plot(1:n,yp)
105
106 %% Write out .wav file
107 filename = "Message_filtered.wav";
108 audiowrite( filename , yp , Fs );
```

6. ANALYZING DIFFERENT FILTERING METHODS, MODIFYING FREQUENCIES, AND DIFFICULTIES ENCOUNTERED

In analyzing the signal we used different types of filters to see how they would affect the overall signal. The types of filters used were:

- Standard Low-pass filter
- Gaussian Filter
- Wavelet Basis Denoising
- FFT filtering,

which were all built-in MATLAB functions except for the Gaussian Filter. The outputs of each were very similar, if not the same. The only significantly different output was that of the standard low-pass filter. The results are below in Figure 11, in the order in which they are mentioned.

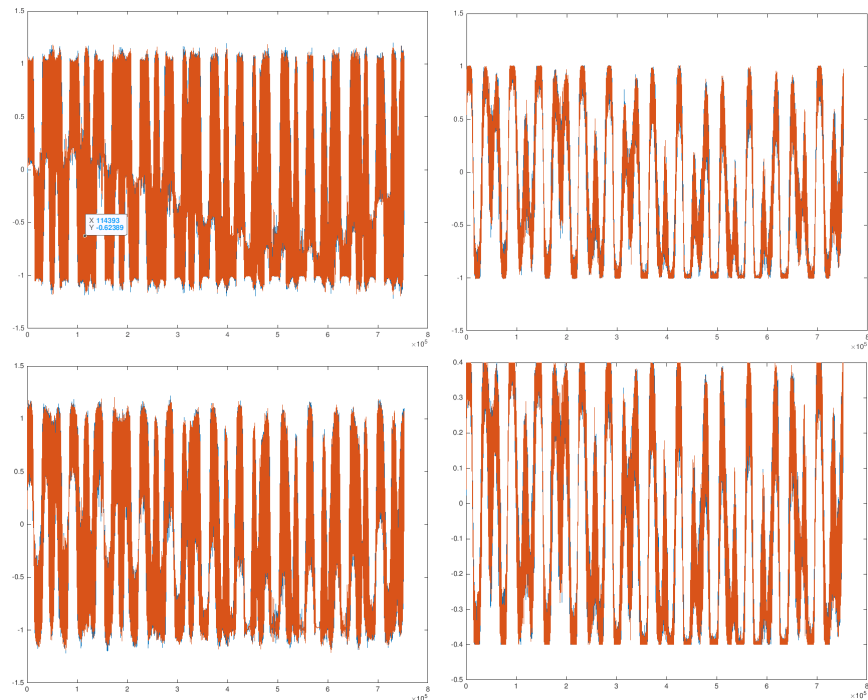


Figure 11

In playback, the Gaussian filter produced the best results, for we were able to discern the following:

“.....do not do that.....r-.....h-...DO NOT-,”

of course over the length of the filtered message. Additionally, we attempted frequency modulation to see if we were able to discern other patterns in the noise. At first, we annihilated parts of the signal to see what would happen, but this only left us with half of the signal. One thing that was interesting was that, after doing this, the signal contained the above message, but looped. So, we decided to amplify some of the frequencies and scaled down other frequencies in order to observe what happened. What we found was that in listing to the signal with the modified frequencies, we heard at different times the above message was playing throughout the entire signal. The code is below.

```
1 %% Project 2
2 clear;clc;clear all;close all;
3
4 % load message
5 file = load('Message.mat');
6 y = file.data;
7 Fs = file.fs;
8 n = length(y);
9 nn = 1:n;
10
11 % DFT and frequency range
12 dft = fft(y);
13 f1 = (0:n-1)*(Fs/n);
14 f1_range = find(f1>100 & f1<1000);
15
16 % Gaussian Filter
17 cconv = @(a,b) real(ifft(fft(a).*fft(b)));
18
19 normalize = @(h) h/sum(h(:));
```

```
20 t = [0:n/2-1, -n/2:-1]';
21 h = @(mu) normalize(exp(-(t.^2.25)/(2*mu^2)));
22 mu = 5;
23
24 sigma = 1;
25 denoise = @(x,mu) cconv(h(mu), x);
26 denoised = denoise(y,mu);
27
28 % Standard lowpass filter
29
30 denoised1 = lowpass(y,0.75);
31
32 % Truncation and Modification
33 dft = fft(y);
34 dft_mod = zeros(size(dft));
35 dft_mod(f1_range,:) = dft(f1_range,:);
36 dft_mod1 = dft_mod;
37 signal = real(ifft(dft_mod));
38
39 % FFT Filter
40 sigfilt = fftfilt(ones(1,10)/25,y); % signal
41 sigfilt_1 = wdenoise(sigfilt,'DenoisingMethod','Minimax');
42 dftfilt_2 = fft(sigfilt_1);
43 dftfilt_2mod = zeros(size(dftfilt_2));
44 dftfilt_2mod(f1_range,:) = dftfilt_2(f1_range,:);
45 dftfilt_2mod1 = dftfilt_2mod(:,1);
46 dftfilt_2mod2 = dftfilt_2mod(:,2);
47
48 % Signals
49 signal_1 = imag(ifft(dftfilt_2mod));
50 signal_1mod = real(ifft(dftfilt_2modmod));
51 signal1 = real(ifft(dft_mod1));
52 signal2 = real(ifft(dft_mean));
53
```

```
54 % Last hunnid thou of signal
55 yy_voice = y(722640:752640)';
56 yy_noise = y(652640:722640)';
57 dft_yy_voice = fft(yy_voice);
58 dft_yy_noise = fft(yy_noise);
59 dft_yy_voice_mod = zeros(size(dft_yy_voice));
60 dft_yy_voice_mod(fl_range,:) = dft_yy_voice(fl_range,:);
61
62 for ii = 1:1708
63     if dft_yy_voice_mod(ii) ≥ 500
64         dft_yy_voice_mod(ii) = ;
65     end
66 end
67
68 for ii = 1706:17068
69     if dft_mod(ii) ≥ dft_yy1_noise_mod(ii)
70         dft_mod(ii) = 0.25*dft_mod(ii);
71     end
72 end
73
74 % Convolution of the Signal
75
76 sig_conv = conv2(y,y,'same');
77 dft_conv = fft(sig_conv);
78
79 max_conv = max(sig_conv);
80 max_sig1 = max(signall);
81 factor = max_sig1./max_conv;
82 scale_sig_conv = factor.*sig_conv;
```

Difficulties

There were few minor difficulties that we encountered along the way. In taking into account everything discussed in the previous sections, the primary difficulty was the noise. There were several interesting features that the noise contained that we noticed:

- the noise could have been an envelope containing highly oscillatory waves
- the noise was indeed rhythmic
- the noise existed at all frequencies, not just in some specific range
- when modulating the frequencies, we noticed the signal playing at different times
- the noise had to be similar to the frequency of the original signal
- despite using different types of filters, the noise persisted.

In taking all of these into account, we conjecture that this noise was no ordinary white noise. Because the noise existed at all frequencies, we could not think of a way to isolate the noise and remove it from the signal. Additionally, we could not determine a way to remove the noise without removing a decent part of the signal. Thus, we conjecture that the noise was made from the original signal. This would explain why it was extremely difficult to remove the noise.

7. CONCLUSION

To conclude, we were able to successfully filter the signal using a low-pass filter. Further, we successfully analyzed the signal from the point of view of understanding what the noise is and how it affects the signal. Despite not being able to fully denoise the signal, we have all gained a greater understanding of signal processing, the use of Discrete Fourier Transforms, convolutions, and how to filter specific frequencies of noise.